MarsOps

User Stories & LoFi Mockups

MarsOps is a distributed automation platform designed to guarantee the survival of occupants in a Martian habitat by integrating heterogeneous IoT devices into a unified monitoring system.

User Stories Overview

Frontend / Operator

US11 Sensor values updated automatically
US12 Out-of-range visual highlighting
US13 Sensor trend line chart
US14 Create, view, delete rules from dashboard
US15 System connection status indicator
US16 Retrieve latest sensor values on load
US17 View actuator ON/OFF state
US18 Manually toggle actuators
US19 Actuator state updates immediately

Backend / System

US1 Periodically poll REST sensors
US2 Receive telemetry data in real time
US3 Normalize raw payloads to standard event
US4 Publish normalized events to broker
US5 Fetch sensors and topics at startup
US6 Create automation rules
US7 Evaluate rules on every broker event
US8 Rules survive service restarts
US9 List configured rules
US10 Delete existing rules
US20 Keep last known actuator state in memory

Groups Page — Sensor Groups Overview

#	As a	I want to	So that
5	Operator	fetch the list of sensors at startup	no manual configuration is required

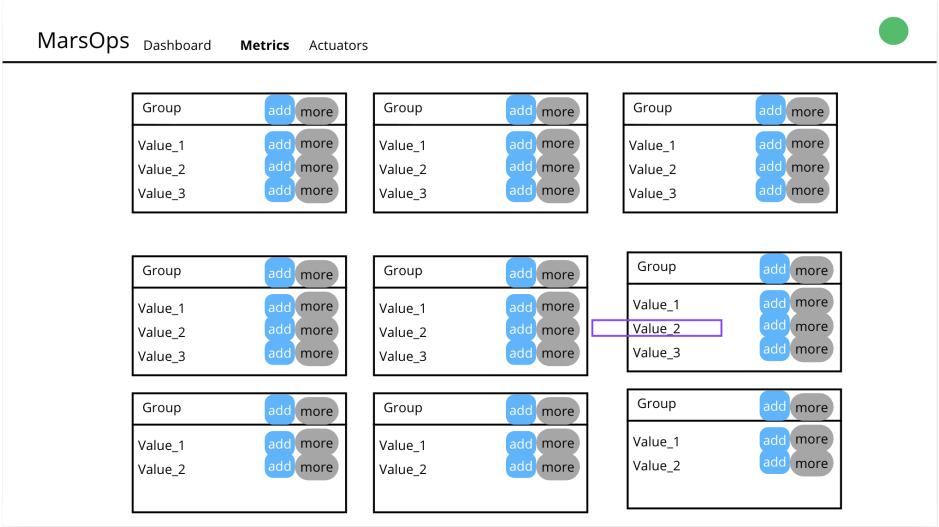
#	As a	I want to	So that
11	Operator	see all sensor values updated automatically	I can monitor the habitat without refreshing the page

#	As a	I want to	So that
16	Frontend	retrieve the latest value of all sensors	the dashboard can be populated on initial load

Non-functional Requirements

Real-time: values pushed via WebSocket, no manual page refresh required
Availability: groups populated at startup by querying the Frontend State API

Groups Page — Sensor Groups Overview — Mockup



Groups Page — Group Detail

#	As a	I want to	So that
11	Operator	see all sensor values updated automatically	I can monitor the habitat without refreshing the page

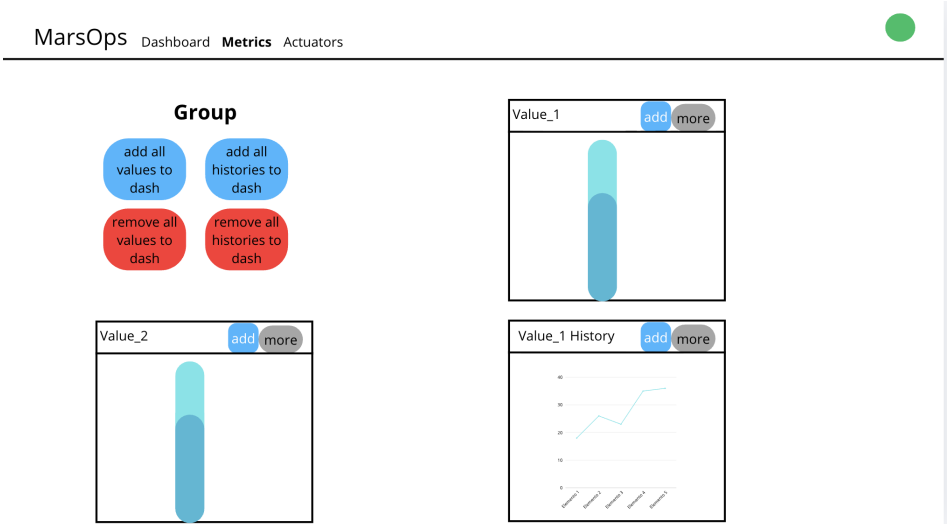
#	As a	I want to	So that
12	Operator	have out-of-range values highlighted visually	I can immediately identify dangerous situations

#	As a	I want to	So that
13	Operator	see a line chart of a sensor's trend during the session	I can spot patterns and anomalies over time

Non-functional Requirements

Visual feedback: the `status` field in the broker event triggers color change (red/orange)
History: trend data retained for the current session only (in-memory)

Groups Page — Group Detail — Mockup



Dashboard — Preferred Values

#	As a	I want to	So that
11	Operator	see all sensor values updated automatically	I can monitor the habitat without refreshing the page

#	As a	I want to	So that
12	Operator	have out-of-range values highlighted visually	I can immediately identify dangerous situations

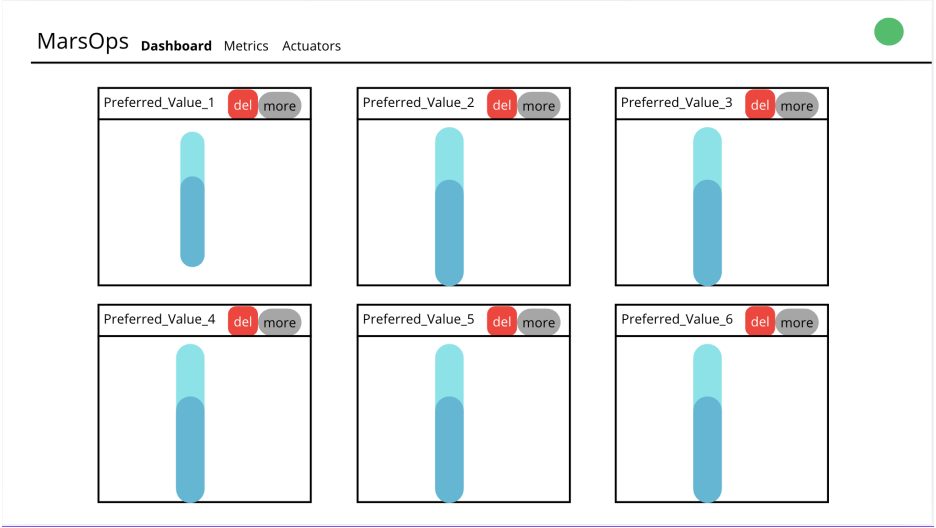
#	As a	I want to	So that
13	Operator	see a line chart of a sensor's trend during the session	I can spot patterns and anomalies over time

#	As a	I want to	So that
15	Operator	see a status indicator showing whether the system is connected	I can immediately tell if the infrastructure is working

Non-functional Requirements

Latency: dashboard reflects new values within seconds of broker publish
Status indicator: green dot (top-right) reflects live WebSocket connection state

Dashboard — Preferred Values — Mockup



Actuators Page — State & Control

#	As a	I want to	So that
17	Operator	see the current ON/OFF state of all actuators	I know what is active in the habitat

#	As a	I want to	So that
18	Operator	manually toggle an actuator on or off	I can intervene directly in emergency situations

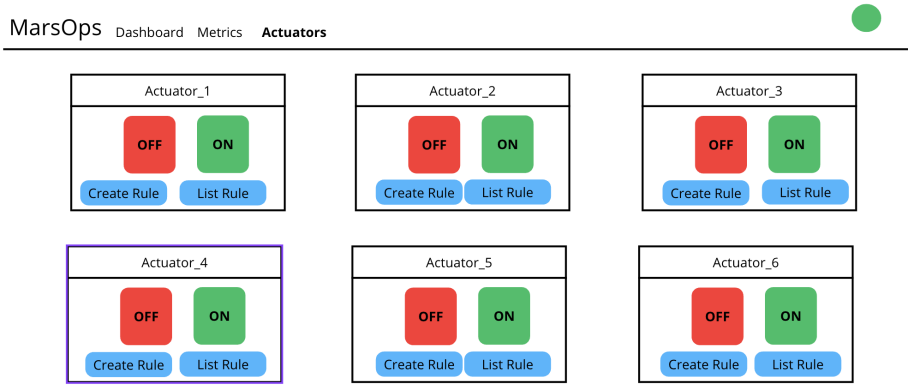
#	As a	I want to	So that
19	Operator	have the actuator state update immediately after every command	I always have a consistent view

#	As a	I want to	So that
20	System	keep the last known state of each actuator in memory	the dashboard and rule engine can access it without querying the environment

Non-functional Requirements

Consistency: toggle sends command to broker; state update propagated back via `actuator.states`
Responsiveness: UI reflects new state without full page reload

Actuators Page — State & Control — Mockup



Actuators Page — Rule List

#	As a	I want to	So that
8	Operator	have rules survive service restarts	I do not have to reconfigure them every time

#	As a	I want to	So that
9	Operator	see the list of all configured rules (condition, target actuator, status)	I can keep track of what is automated

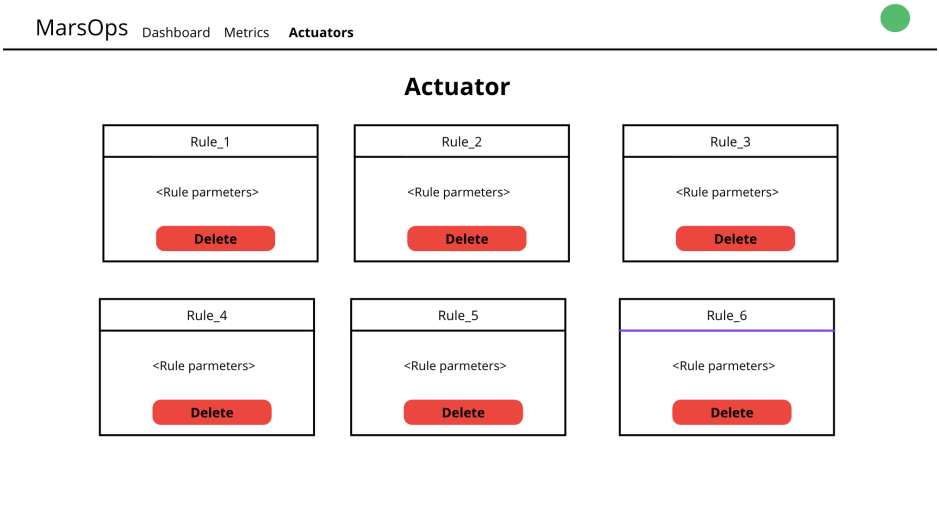
#	As a	I want to	So that
10	Operator	delete an existing rule	I can remove automations that are no longer needed

#	As a	I want to	So that
14	Operator	view rules directly from the dashboard	I do not need external tools

Non-functional Requirements

Persistence: rules stored in SQLite and survive service restarts
Scoped view: rule list is filtered per actuator for easier management

Actuators Page — Rule List — Mockup



Actuators Page — Create Rule

#	As a	I want to	So that
6	Operator	create rules so that actuators respond automatically to habitat conditions	dangerous environmental shifts are handled without manual intervention

#	As a	I want to	So that
7	System	evaluate all relevant rules on every incoming broker event	actuators are commanded in real time when conditions are met

#	As a	I want to	So that
14	Operator	create rules directly from the dashboard	I do not need external tools

Rule Model

`IF` *sensor operator value* `THEN set` *actuator* `to` *state*

Operators: $>, <, \geq, \leq, =$ **Evaluated in real-time** on every `sensor.events` message published to the broker

Actuators Page — Create Rule — Mockup

MarsOps Dashboard Metrics **Actuators**

Actuator

IF *sensor* *operator* *value*

THEN set

actuator to *state*

Discard Rule **Save Rule** *<Rule_name>*

Backend — Converter Service

#	As a	I want to	So that
1	Operator	have the system periodically poll all REST sensors	data is continuously updated without manual intervention

#	As a	I want to	So that
2	Operator	have the system receive all telemetry data in real time	

#	As a	I want to	So that
3	Developer	have every raw payload converted into a standard internal event	downstream components work on homogeneous data

#	As a	I want to	So that
4	Ingestion component	have every normalized event published to a message broker topic	all services can consume it in a decoupled way

#	As a	I want to	So that
5	Operator	have the system fetch sensors and topics at startup	no manual configuration is required

Non-functional Requirements

Polling interval: 5s for REST sensors **SSE:** auto-reconnect on disconnect

Backend — Automation Rules Service

#	As a	I want to	So that
6	Operator	create if-then rules via REST API	actuators respond automatically to habitat conditions

#	As a	I want to	So that
7	System	evaluate rules on every broker event	actuators are commanded in real time when conditions are met

#	As a	I want to	So that
8	Operator	have rules survive service restarts	I do not have to reconfigure them every time

#	As a	I want to	So that
9	Operator	see the list of all configured rules	I can keep track of what is automated

#	As a	I want to	So that
10	Operator	delete rules via REST API	I can remove automations that are no longer needed

#	As a	I want to	So that
20	System	keep the last known actuator state in memory	dashboard and rule engine can access it without querying the environment